

Phase 0 – Environment Setup for MMO-Lite Action MMORPG (UE 5.8)

Audience: Claude Code, acting as setup engineer on the developer's machine.

Goal: Install and verify the full local dev toolchain per the project's build docs (UE 5.8 C++ + GAS, Nakama + PostgreSQL + Redis via Docker, Git + LFS with locking).

Rule: Do each step in order. After each step, run its verification. Do not proceed past a failed verification – report it and stop.

0. Decisions (locked defaults + required fill-ins)

Decision	Value	Status
Project codename	__FILL_IN__	REQUIRED before repo creation – ask the developer.

Decision	Value	Status
		Used for .uproject name, repo name, C++ module prefix. Hard to rename later.
First two classes	__FILL_IN__ (1 melee + 1 ranged)	Required before Phase 1, not blocking setup
MVP class count	2	Locked (default)
Nakama runtime	TypeScript	Locked (default)
Tick rate	30 Hz world / 60 Hz arena	Locked (default)
Lyra usage	Borrow patterns; single-plugin structure now	Locked (default)
EAC	Defer to pre-public-beta	Locked (default)
Nakama hosting	Self-host later; Docker locally now	Locked (default)

Assumption: Windows 11 dev machine, developer has admin rights. If the OS is not Windows, stop and report.

1. Preflight hardware check (automate)

Run and report; warn if below threshold, hard-stop only on disk:

- **Disk:** ≥ 250 GB free on the drive that will hold the engine source build. **Hard stop if < 150 GB.**
- **RAM:** ≥ 32 GB recommended (16 GB workable but slow builds — warn only).
- **CPU cores:** report count (affects engine compile time, no gate).
- Report GPU model (informational).

PowerShell one-liners are fine (`Get-PSDrive` , `Get-CimInstance Win32_ComputerSystem` , `Win32_Processor` , `Win32_VideoController`).

2. Manual steps — human required (list these to the developer, do not attempt)

Claude Code cannot do these (accounts / GUI / license acceptance):

1. **Epic Games account** — create/sign in.
2. **Link GitHub account to Epic** at epicgames.com → account → Apps & Accounts, then accept the invite to the `EpicGames` GitHub org. Required for engine **source** access (dedicated-server targets cannot be built on the launcher engine).
3. **Epic Games Launcher sign-in** after install (install itself is automated below).
4. **Docker Desktop**: accept license and enable WSL2 backend on first launch.
5. **AWS account** — needed later for GameLift; not blocking Phase 0. Skip for now.

3. Automated installs (winget)

Run each; verify after:

```
winget install --id Git.Git -e --accept-  
package-agreements --accept-source-  
agreements  
winget install --id GitHub.GitLFS -e  
winget install --id Docker.DockerDesktop -e  
winget install --id  
EpicGames.EpicGamesLauncher -e  
winget install --id  
Microsoft.VisualStudio.2022.Community -e --
```

```
override "--add  
Microsoft.VisualStudio.Workload.NativeGame  
--add  
Microsoft.VisualStudio.Workload.NativeDesktop  
op --includeRecommended --passive"  
winget install --id OpenJS.NodeJS.LTS -e  
# for Nakama TypeScript runtime tooling
```

Then:

```
git lfs install
```

Verify:

- `git --version`, `git lfs version`, `node --version`, `npm --version` all return.
- Visual Studio: confirm "Game development with C++" workload present (`vswhere -products * -requires Microsoft.VisualStudio.Workload.NativeGame`).
- Docker: `docker --version`; `docker run hello-world` succeeds (may require the manual first-launch step in §2.4 first).

Note: Also install the **Visual Studio Tools for Unreal Engine** component if not pulled in by the workload (`Microsoft.VisualStudio.Component.Unreal` via VS Installer modify).

4. Unreal Engine 5.8 — two-track install

Track A (today, unblocks Phase 1 single-player work):

Instruct the developer to install **UE 5.8 from the Epic Launcher** (GUI, manual). Verify by launching the editor and creating a throwaway Blank **C++** project that compiles.

Track B (parallel, required for dedicated servers):

After §2.2 is done, clone and build engine source. Confirm with the developer which drive to use (§1 disk check), then:

```
git clone --depth 1 -b 5.8
https://github.com/EpicGames/UnrealEngine.g
it D:\UE58src
cd D:\UE58src
.\Setup.bat
.\GenerateProjectFiles.bat
# Build via MSBuild (Development Editor |
Win64) — this takes 1-4+ hours:
& "C:\Program Files\Microsoft Visual
Studio\2022\Community\MSBuild\Current\Bin\M
SBuild.exe" UE5.sln /t:Build
/p:Configuration="Development Editor"
/p:Platform=Win64 /m
```

Also install the **Linux cross-compile toolchain** matching UE 5.8 (Epic's clang toolchain installer —

find the exact version in the 5.8 release notes / Linux Development docs and give the developer the link if it's a GUI installer).

Verify: source-built editor launches; Development Server | Win64 target of a test project compiles.

Acceptance for task 0.1: editor launches AND a blank C++ project builds. Server target build can complete later without blocking §5–§7.

5. Project repo setup (task 0.2)

Two-repo rule: engine source (Track B) is NOT the project repo. Create a separate clean repo named after the codename. Claude Code and Codex only ever operate inside the project repo.

1. `git init <Codename>` (get codename from developer first — see §0).
2. Add UE `.gitignore`: ignore `Binaries/`, `Intermediate/`, `Saved/`, `DerivedDataCache/`, `.vs/`, `*.sln`.
3. Add `.gitattributes` with LFS + locking for UE binaries:

```
*.uasset filter=lfs diff=lfs merge=binary -
text lockable
*.umap filter=lfs diff=lfs merge=binary -
```

```
text lockable
*.png    filter=lfs diff=lfs merge=binary -
text
*.fbx    filter=lfs diff=lfs merge=binary -
text
*.wav    filter=lfs diff=lfs merge=binary -
text
```

4. `git lfs track` confirms patterns; commit both files as the first commit.
5. Ask the developer which remote host to use.
Recommend **Azure DevOps** (free unlimited LFS) over GitHub free (LFS quota: 1 GB storage / 1 GB bandwidth per month — too small for UE). Set up remote + push.
6. Set up a nightly mirror/backup of the remote (simple `git clone --mirror` scheduled task is acceptable; locking protects against conflicts, not loss).

Acceptance: repo pushes to remote; `git lfs locks` command works against the remote (locking verified once the in-editor Git plugin is added later — note as TODO, don't block).

6. Local backend stack: Nakama + PostgreSQL + Redis (task 0.3)

Create `backend/docker-compose.yml` in the project repo:

```
services:
  postgres:
    image: postgres:15-alpine
    environment:
      POSTGRES_DB: nakama
      POSTGRES_PASSWORD: localdev
    volumes:
      - pgdata:/var/lib/postgresql/data
    ports:
      - "5432:5432"
    healthcheck:
      test: ["CMD", "pg_isready", "-U",
"postgres", "-d", "nakama"]
      interval: 5s
      timeout: 5s
      retries: 10

  nakama:
    image:
registry.heroiclabs.com/heroiclabs/nakama:3
    entrypoint:
      - "/bin/sh"
      - "-ecx"
      - >
        /nakama/nakama migrate up --
database.address
postgres:localdev@postgres:5432/nakama &&
      exec /nakama/nakama --name nakama1
      --database.address
postgres:localdev@postgres:5432/nakama
```

```
    --logger.level DEBUG
    --session.token_expiry_sec 7200
  depends_on:
    postgres:
      condition: service_healthy
  ports:
    - "7349:7349"
    - "7350:7350"
    - "7351:7351"

  redis:
    image: redis:7-alpine
    ports:
      - "6379:6379"

  volumes:
    pgdata:
```

Run `docker compose up -d` from `backend/`.

Verify (acceptance for 0.3):

- `docker compose ps` shows all three healthy/running.
- Nakama console reachable at `http://127.0.0.1:7351` (default login `admin` / `password` — note to developer: fine locally, never in cloud).
- `curl http://127.0.0.1:7350/` returns a Nakama response.
- `docker exec` into redis: `redis-cli ping` → `PONG`.

Schema rule (record in repo docs): every Nakama storage object and Postgres table we create later MUST carry a `schema_version` integer field from day 1.

7. Nakama Unreal SDK + Sentry (task 0.4)

1. Download the **Nakama Unreal client** plugin (Heroic Labs GitHub, release matching UE 5.8 — if no 5.8 build exists yet, take latest and note it must compile from source into `Plugins/`).
2. Place in `<Project>/Plugins/Nakama/` . Commit.
3. Sentry: register free tier (manual, human), then add the **Sentry Unreal plugin** to `Plugins/` . If the developer skips Sentry today, mark as deferred — not blocking.

Acceptance: project compiles with Nakama plugin enabled; a test C++ or Blueprint call performs **device authentication** against `127.0.0.1:7350` and the session appears in the Nakama console.

(The auth test can slip to the start of Phase 1 if the UE project isn't created yet — record as first Phase 1 subtask.)

8. Standards docs + task board (task 0.5)

Create in the project repo:

- `docs/adr/0001-asc-placement.md` — ADR template + first ADR: *AbilitySystemComponent lives on PlayerState for players, on Pawn for AI* (Lyra pattern).
- `docs/standards.md` — record: Epic C++ Coding Standard; C++ for systems/networking/authority, Blueprint/DataAssets for content and tuning; Gameplay Tag taxonomy (`Ability.*` , `State.*` , `Damage.Type.*` , `Cooldown.*` , `Event.Montage.*`); folder naming conventions; the `schema_version` rule from §6.
- Task board: create `docs/backlog.md` seeded with Phase 0 (0.1–0.5) and Phase 1 (1.1–1.5) tasks from the build roadmap, each with priority, dependencies, and "done when" criteria. (External tool like GitHub Projects optional — file is the minimum.)

Acceptance: all three files committed and pushed.

9. Final report

When done, output a table: step | status (done / blocked / deferred) | verification result | notes. List every manual step still owed by the developer (§2 items, launcher install, Sentry account, codename if still unset).

Out of scope for this run

GameLift/AWS, EAC, Replication Graph, Perforce migration, any gameplay code. Do not install these.